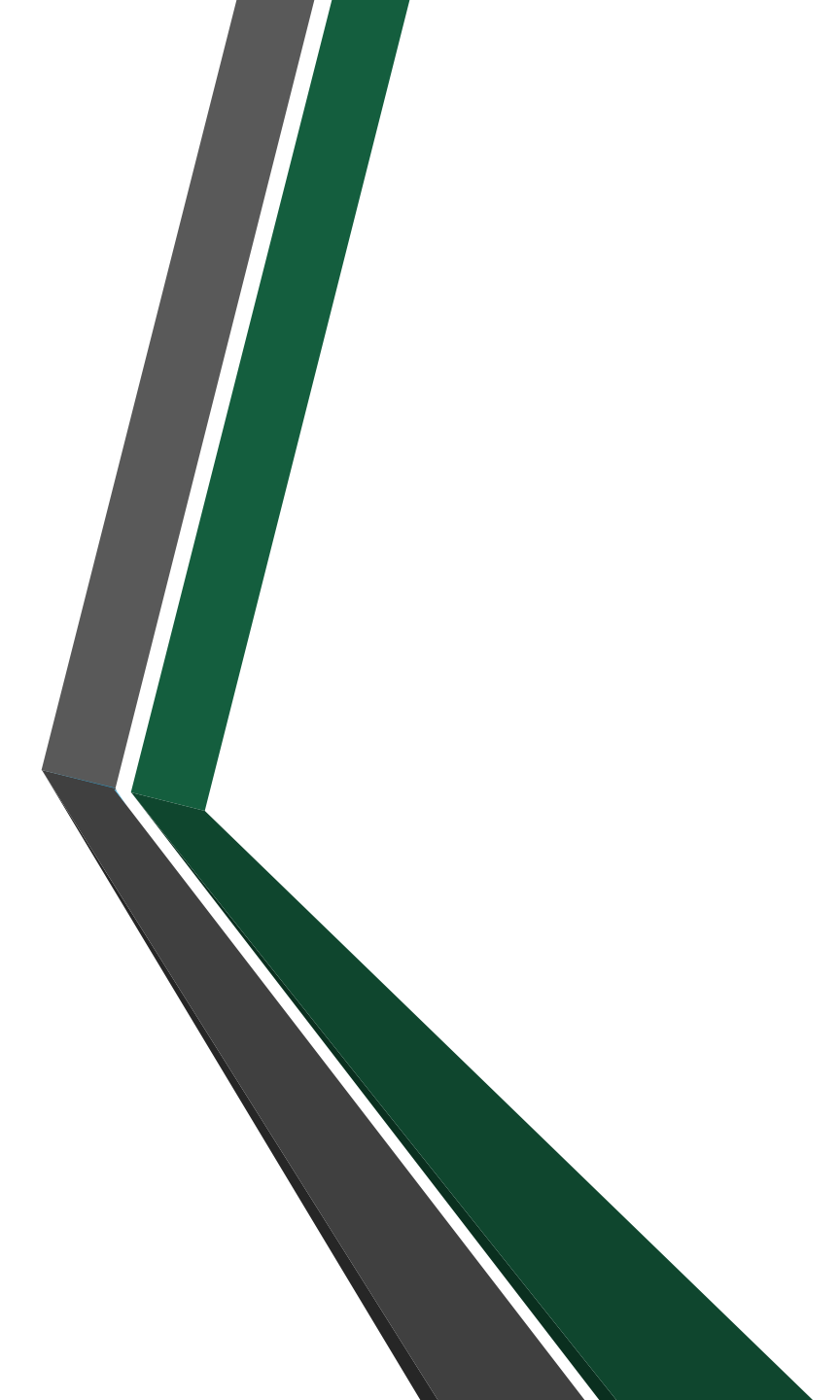




BAZE DE DATE

CURS 8

Evaluarea și optimizarea interogărilor

Procesarea interogărilor

- O **expresie a algebrei relaționale** este constituită din **relații** legate prin **operații** din algebra relațională;
- O expresie se poate reprezenta **grafic** cu ajutorul unui **arbore**, numit **arbore algebric**, în care nodurile corespund operatorilor din cadrul expresiei respective;
- Evaluarea unei expresii presupune efectuarea prelucrărilor indicate de operatorii din expresie în ordinea aparițiilor sau în ordinea fixată prin paranteze;
- **Rezultatul** evaluării unei expresii este o **relație derivată** din relațiile menționate ca operanzi în cadrul expresiei;
- Două expresii sunt echivalente, dacă în urma evaluării lor se obține ca rezultat aceeași relație;

Procesarea interogărilor

- În majoritatea sistemelor de gestiune, în special în cele relaționale, interfața cu utilizatorul este de tip **neprocedural**;
 - Utilizatorul definește datele pe care dorește să le vizualizeze fără a da algoritmi de acces (**CE**, nu **CUM**)
 - Sistemul trebuie să convertească cererea utilizatorului:
 - într-o cerere optimală;
 - în proceduri de acces optimal la datele fizice;
- Garantarea absolută a performanțelor optime pentru procesorul limbajului relațional este imposibilă;
- Corectă ar fi utilizarea cuvântului „**ameliorare**” în locul cuvântului „**optimizare**”;

Procesarea interogărilor

În momentul în care se implementează o cerere (interogare) în SQL, sistemul de gestiune (Oracle Database) evaluează cererea.

Evaluarea unei interogări se efectuează în trei etape:

- **Analiza cererii** – presupune **studierea semantică și sintactică** a cererii pentru a verifica corectitudinea sa și a simplifica criteriul de căutare.

Semantic – se verifică dacă programul rulează și compilează **fără erori**. **Sintactic** – se verifică dacă programul rulează și compilează fără erori, dar și dacă este corectă sintaxa utilizată (pentru a verifica corectitudinea cererii). În acest moment se verifică și accesul la datele implicate în cererea SQL.

Procesarea interogărilor

- **Ordonanțarea/Ordonarea** – presupune **descompunerea cererii într-o mulțime de operații elementare** și determinarea unei ordini optime a acestor operații.

Operațiile sunt, în general, cele ale **algebrei relaționale** (SELECT, PROJECT, JOIN, DIVISION, UNION, INTERSECT, PRODUCT, DIFFERENCE), după care se stabilește o ordine de execuție optimă a acestor operații.

La sfârșitul etapei se obține un **plan de execuție al cererii**.

Procesarea interogărilor

- **Execuția** – presupune **efectuarea** (paralelă și/sau secvențială) operațiilor elementare furnizate de planul de execuție pentru a obține **rezultatul cererii**.

Procesarea interogărilor

- Presupunem că utilizatorul transmite sistemului de gestiune o **cerere** exprimată prin **comenzi SQL**
- Pentru a răspunde cererii, SGBD-ul trebuie să înțeleagă cererea utilizatorului
- Cererea trebuie să:
 - fie corectă sintactic;
 - aibă **datele disponibile** utilizatorului;
 - localizeze datele analizând diferite **drumuri de acces** la ele;

Procesarea interogărilor

- Aceste funcții sunt realizate de SGBD cu ajutorul a **două module funcționale** care comunică permanent:
 - **analizorul cererilor**, care asigură:
 - verificarea **sintactică** și **semantică** a cererii;
 - localizarea datelor implicate în cerere (găsirea adresei blocurilor ce conțin datele - verifică unde se află efectiv datele pe disc);
 - furnizarea planului de execuție - decide cea mai eficientă metodă de a extrage datele;
 - **administratorul datelor** (executorul – execută fizic cererea), care execută efectiv cererea (primește planurile de execuție furnizate de modulul de optimizare și le execută)
 - Execuția presupune căutarea blocurilor ce conțin datele și transferul blocurilor în memoria *cache*;

Procesarea interogărilor

Transferul în memoria cache:

- Datele **NU** sunt citite direct de fiecare dată de pe disc – pentru că este lent;
- În schimb, **sunt copiate în memoria cache (buffer pool)**, care este o zonă rapidă din RAM, gestionată de SGBD;

Memoria cache acționează ca un **intermediar rapid** între stocarea lentă (pe disc) și procesorul care are nevoie de date;

Dacă aceeași cerere (sau una similară) se execută din nou mai târziu:

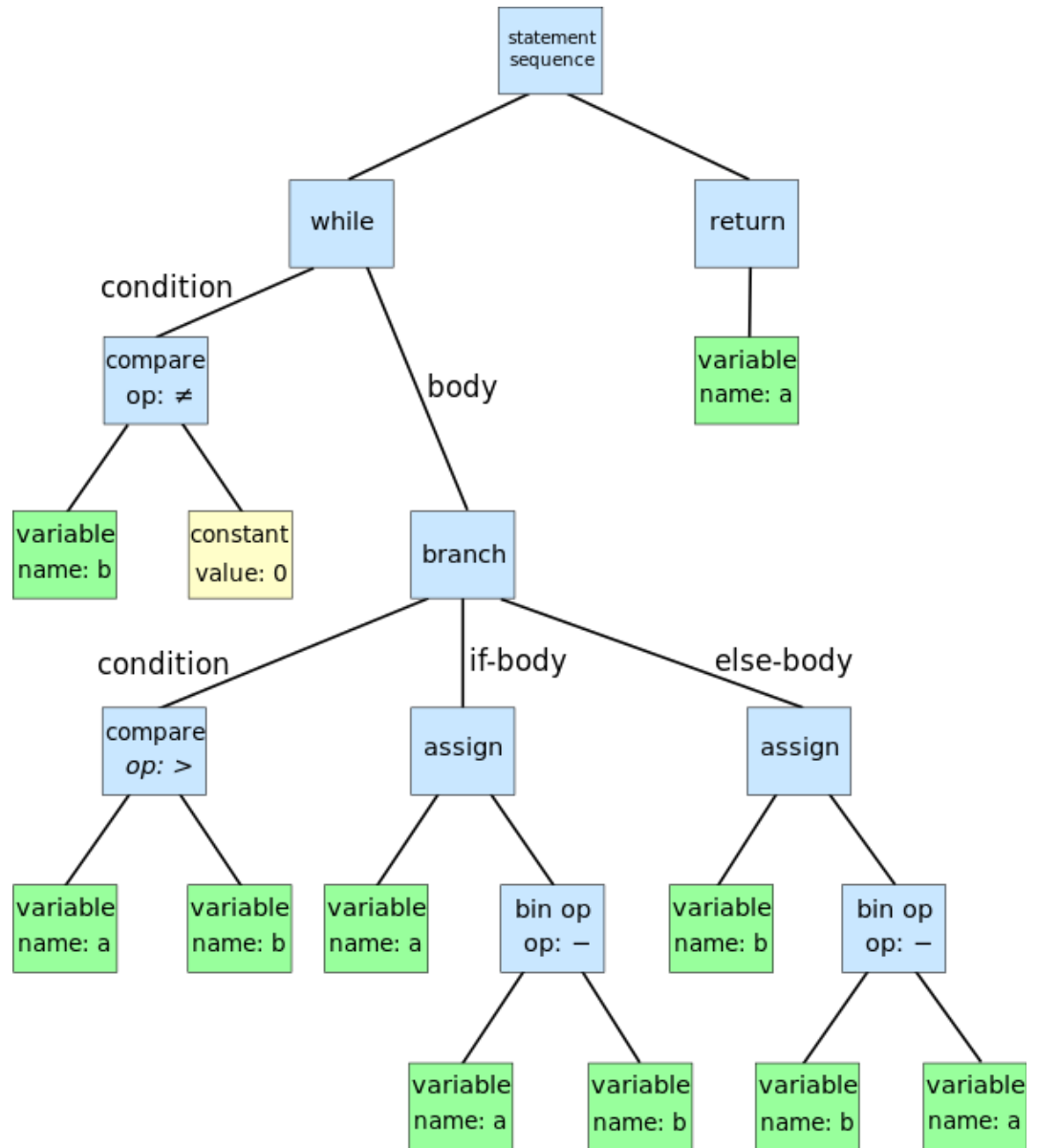
- Datele fiind deja în cache, vor fi returnate instantaneu, fără citire de pe disc
→ crește **viteza de răspuns**.

Procesarea interogărilor

- În general, compilatorul efectuează trei tipuri de analiză: **lexicală, sintactică, semantică**
- Se verifică dacă programul rulează și compilează fără erori. **Erorile semantice** sunt erorile de compilare care nu sunt erori de sintaxă (ex: nu avem tipuri de date compatibile, nu se realizează o conversie automată, nu are acces la o proprietate, etc).
- În final, pentru realizarea tuturor verificărilor, compilatorul realizează un arbore, după cum se observă în exemplul următor:

Procesarea interogărilor

```
while b  $\neq$  0  
  if a > b  
    a = a - b  
  else  
    b = b - a  
return a
```



Procesarea interogărilor

- Compilatorul SQL execută o cerere într-un mod similar, convertind cererea utilizatorului într-o variantă optimizată, cu ajutorul unui arbore algebric;

Ideea generală:

cerere → arbore algebric (nu este unic) → plan de execuție → optimizare

Procesarea interogărilor

- Cu ajutorul **planului de execuție**, se realizează o evaluare a cererii, în vederea realizării unor optimizări;
- **Arborele algebric** nefiind unic, pot exista planuri de execuție diferite, dar care în final să producă același rezultat;
- **Scopul optimizării** este acela de **a identifica planul de execuție optim** (de cost minim);
- Un plan de execuție implică:
 - **o secvență de pași pentru evaluarea cererii** - în mod obișnuit, fiecare pas din planul de execuție corespunde unei **operații relaționale**;
 - **metoda care va fi folosită pentru evaluarea operației** - de obicei, pentru o operație relațională dată, există mai multe metode ce pot fi folosite pentru evaluarea acesteia;

Procesarea interogărilor

- Două planuri de execuție diferite, care au întotdeauna același rezultat, se numesc **echivalente**. Planuri de execuție echivalente pot avea diferite costuri;
- Scopul optimizării cererilor este de a găsi, printre diversele planuri de execuție echivalente, pe acela de cost minim;
- Într-un sistem centralizat, costul evaluării unei cereri este suma a două componente:
 - **costul I/O** (transferuri de date) + **costul CPU** (verificare de condiții, operații *join*, etc.)

Ordinea de execuție a operațiilor

- O interogare constă dintr-un număr de operații
 - ordinea în care se efectuează operațiile are un rol important în evaluarea costului necesar realizării interogării;
- Există **două modalități** de abordare pentru a determina ordinea de execuție a operațiilor:
 - algebric;
 - bazat pe estimarea costului;
- Ambele folosesc o mulțime de **reguli** care permit **transformarea unui plan de execuție** (reprezentat ca o expresie scrisă în termenii algebrei relaționale) în altul, echivalent

Ordinea de execuție a operațiilor

- Astfel, o **cerere SQL** se poate scrie sub forma unei **expresii a algebrei relaționale**, formată din relații și operații specifice algebrei relaționale. Ulterior, aceasta se poate reprezenta grafic sub forma unui arbore, numit **arbore algebric**, în care nodurile corespund operatorilor algebrei relaționale utilizați în cadrul cererii, urmând etapa de evaluare și optimizare a acesteia;
- Optimizarea cererilor, utilizând algebra relațională, se realizează pornind de la scrierea cererii sub forma unor **expresii algebrice**, după care se aplică transformări echivalente, dar mai eficiente;
- Pentru optimizare se utilizează și **proprietățile operatorilor algebrei relaționale**;

Proprietățile operatorilor algebrei relaționale

Proprietățile sunt explicate detaliat, fiind însoțite de exemple, în documentul:

Curs_8_Proprietati_Operatorii_Exemple

Proprietățile operatorilor algebrei relaționale

Proprietatea 1. Comutativitatea operațiilor de *join* și produs cartezian:

$$\text{JOIN}(R1, R2) = \text{JOIN}(R2, R1)$$

$$R1 \times R2 = R2 \times R1$$

Proprietatea 2. Asociativitatea operațiilor de *join* și produs cartezian:

$$\text{JOIN}(\text{JOIN}(R1, R2), R3) = \text{JOIN}(R1, \text{JOIN}(R2, R3))$$

$$(R1 \times R2) \times R3 = R1 \times (R2 \times R3)$$

Proprietatea 3. Compunerea proiecțiilor:

$$\Pi_{A1, \dots, Am} (\Pi_{B1, \dots, Bn} (R)) = \Pi_{A1, \dots, Am} (R),$$

unde $\{A_1, A_2, \dots, A_m\} \subseteq \{B_1, B_2, \dots, B_n\}$.

Proprietatea 4. Compunerea selecțiilor:

$$\sigma_{cond1} (\sigma_{cond2} (R)) = \sigma_{cond1 \wedge cond2} (R) = \sigma_{cond2} (\sigma_{cond1} (R)),$$

unde am notat prin *cond* condiția după care se face selecția.

Proprietățile operatorilor algebrei relaționale

Proprietatea 5. Comutarea selecției cu proiecția:

$$\Pi_{A_1, \dots, A_m} (\sigma_{cond} (R)) = \sigma_{cond} (\Pi_{A_1, \dots, A_m} (R)),$$

unde condiția după care se face selecția implică numai attributele $A_1, \dots, A_m$.

Dacă condiția implică și attributele $B_1, \dots, B_n$, care nu aparțin mulțimii $\{A_1, \dots, A_m\}$, atunci:

$$\Pi_{A_1, \dots, A_m} (\sigma_{cond} (R)) = \Pi_{A_1, \dots, A_m} (\sigma_{cond} (\Pi_{A_1, \dots, A_m, B_1, \dots, B_n} (R)))$$

Proprietățile operatorilor algebrei relaționale

Proprietatea 6. Comutarea selecției cu produsul cartezian:

Dacă toate atributele menționate în condiția după care se face selecția sunt atribute ale relației $R1$, atunci:

$$\sigma_{cond} (R1 \times R2) = \sigma_{cond} (R1) \times R2$$

Dacă condiția este de forma $cond1 \wedge cond2$ și dacă $cond1$ implică numai atribute din $R1$, iar $cond2$ implică numai atribute din $R2$, atunci

$$\sigma_{cond} (R1 \times R2) = \sigma_{cond1} (R1) \times \sigma_{cond2} (R2)$$

Dacă $cond1$ implică numai atribute din $R1$, iar $cond2$ implică atribute atât din $R1$ cât și din $R2$, atunci:

$$\sigma_{cond} (R1 \times R2) = \sigma_{cond2} (\sigma_{cond1} (R1) \times R2)$$

Proprietățile operatorilor algebrei relaționale

Proprietatea 7. Comutarea selecției cu reuniunea:

$$\sigma_{cond} (R1 \cup R2) = \sigma_{cond} (R1) \cup \sigma_{cond} (R2)$$

Proprietatea 8. Comutarea selecției cu diferența:

$$\sigma_{cond} (R1 - R2) = \sigma_{cond} (R1) - \sigma_{cond} (R2)$$

Proprietatea 9. Comutarea proiecției cu produsul cartezian:

Dacă $A_1, \dots, A_m$ este o listă de atribute ce apar în schemele relaționale $R1$ și $R2$ și dacă lista este formată din atribute aparținând lui $R1$ (notate prin $B_1, \dots, B_n$) și din atribute aparținând lui $R2$ (notate prin $C_1, \dots, C_k$) atunci:

$$\Pi_{A_1, \dots, A_m} (R1 \times R2) = \Pi_{B_1, \dots, B_n} (R1) \times \Pi_{C_1, \dots, C_k} (R2)$$

Proprietatea 10. Comutarea proiecției cu reuniunea:

$$\Pi_{A_1, \dots, A_m} (R1 \cup R2) = \Pi_{A_1, \dots, A_m} (R1) \cup \Pi_{A_1, \dots, A_m} (R2)$$

Proprietățile operatorilor algebrei relaționale

Proprietatea 11. Compunerea proiecției cu operația *join*:

Dacă $A_1, \dots, A_m$ este o listă de atribute ce apar în schemele relaționale $R1$ și $R2$ și dacă lista este formată din atribute aparținând lui $R1$ (notate prin $B_1, \dots, B_n$) și din atribute aparținând lui $R2$ (notate prin $C_1, \dots, C_k$) atunci:

$$\Pi_{A_1, \dots, A_m} (\text{JOIN}(R1, R2, D)) = \Pi_{A_1, \dots, A_m} (\text{JOIN}(\Pi_{D, B_1, \dots, B_n}(R1), \Pi_{D, C_1, \dots, C_k}(R2), D)),$$

unde am notat prin $\text{JOIN}(R1, R2, D)$ operația de compunere naturală între $R1$ și $R2$ după atributul comun D .

Proprietățile operatorilor algebrei relaționale

Proprietatea 12. Compunerea selecției cu operația *join*:

$$\sigma_{\text{cond}} (\text{JOIN} (R1, R2, D)) = \sigma_{\text{cond}} (\text{JOIN} (\Pi_{D,A} (R1), \Pi_{D,A} (R2), D)),$$

unde A reprezintă attributele care apar în condiția după care se face selecția.

Reguli de optimizare frecvent folosite

Regula de optimizare 1:

Selecțiile se execută cât mai devreme posibil. Motivația acestei reguli este că selecțiile reduc substanțial dimensiunea relațiilor. Regula de transformare 4 poate fi folosită pentru a separa două sau mai multe selecții în selecții individuale care pot fi distribuite *join*-ului sau produsului cartezian folosind comutarea selecției cu *join*-ul.

Reguli de optimizare frecvent folosite

Regula de optimizare 2:

Produsele carteziane se înlocuiesc cu *join*-uri, ori de câte ori este posibil.

Un produs cartezian între două relații este de obicei mult mai scump (ca și cost) decât un *join* între cele două relații, deoarece primul generează concatenarea tuplurilor în mod exhaustiv și poate genera un rezultat foarte mare. Această transformare se poate realiza folosind legătura dintre produs cartezian, *join* și selecție.

Reguli de optimizare frecvent folosite

Regula de optimizare 3:

Dacă sunt mai multe *join*-uri atunci cel care se execută primul este cel mai restrictiv

- Un *join* este mai restrictiv decât altul dacă produce o relație mai mică;
- Se poate determina care *join* este mai restrictiv pe baza selectivității sau cu ajutorul informațiilor statistice;
- Algebric, acest lucru se poate realiza folosind regula de transformare 2;

Reguli de optimizare frecvent folosite

Dacă sunt mai multe *join*-uri atunci cel care se execută primul este cel mai restrictiv

Când sunt mai multe **JOIN-uri** într-o cerere SQL, cel mai restrictiv înseamnă acel JOIN **care filtrează cele mai multe rânduri de la început** – adică lasă în rezultat mai puține date de procesat mai departe.

De ce este important?

Pentru că atunci când baza de date execută cererea, ea trebuie să combine tabelele. Dacă începe cu un **JOIN care reduce mult volumul de date**, următoarele operații se vor face pe mai puține rânduri, deci vor fi mai rapide și vor consuma mai puține resurse (memorie, CPU etc.).

Reguli de optimizare frecvent folosite

DE EXEMPLU:

SELECT *

FROM employees e JOIN departments d

ON e.department_id = d.department_id

JOIN jobs j ON e.job_id = j.job_id

WHERE j.job_title = IT_PROG';

JOIN-ul cu **JOBS** este restrictiv pentru că **WHERE j.job_title = IT_PROG'** filtrează datele → este mult mai bine să se înceapă cu **employees** JOIN **jobs** care va păstra mai puține rânduri.

employees JOIN jobs on

JOIN departments ON ...

Reguli de optimizare frecvent folosite

DE EXEMPLU:

SELECT *

FROM employees e JOIN departments d

ON e.department_id = d.department_id

JOIN jobs j ON e.job_id = j.job_id

WHERE j.job_title = IT_PROG';

OBSERVAȚIE!

La INNER JOIN, ordinea în care sunt scrise tabelele în SQL nu obligă serverul să le execute în aceeași ordine.

Optimizerul Oracle poate schimba ordinea lor și poate alege singur arborele de execuție cu cost mai mic.

Reguli de optimizare frecvent folosite

CONCLUZIE:

Cel mai restrictiv JOIN → este acel **JOIN** care reduce cel mai mult numărul de rânduri.

→ Este ideal să se execute primul pentru că reduce volumul de date, accelerează restul cererii și optimizează resursele serverului.

Reguli de optimizare frecvent folosite

Regula de optimizare 4:

Proiecțiile se execută la început pentru a îndepărta attributele nefolositoare.

- Dacă un atribut al unei relații nu este folosit în operațiile ulterioare, atunci trebuie îndepărtat. În felul acesta se va folosi o relație mai mică în operațiile ulterioare;
- Aceasta se poate realiza folosind comutarea proiecției cu *join*-ul;

Arbori algebrici. Simbolurile operatorilor algebrei relaționale.

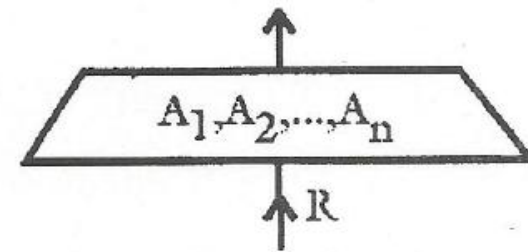
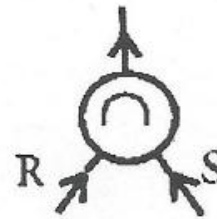
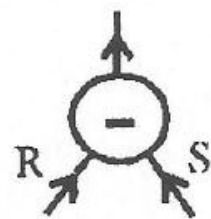
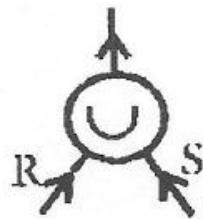
- În continuare, vom studia simbolurile operatorilor algebrei relaționale;
- Cu ajutorul acestora, fiecare operator al algebrei relaționale se poate reprezenta grafic;
- Reprezentarea grafică o să formeze, în final, arborele algebric asociat expresiei algebrice;

UNION

DIFFERENCE

INTERSECT

PROJECT

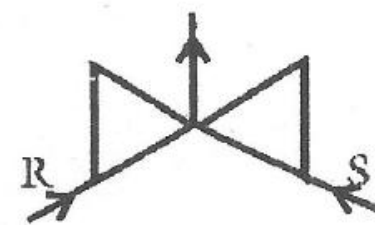
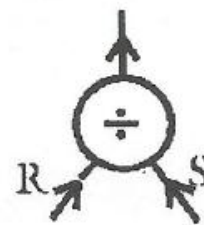
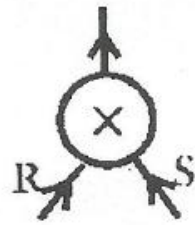
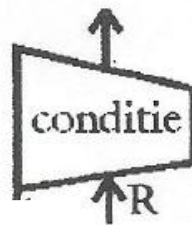


SELECT

PRODUCT

DIVISION

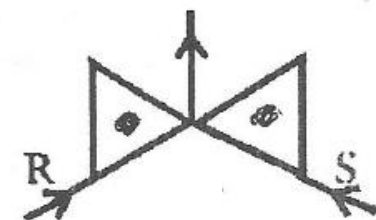
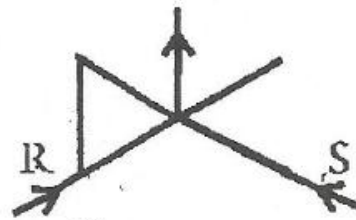
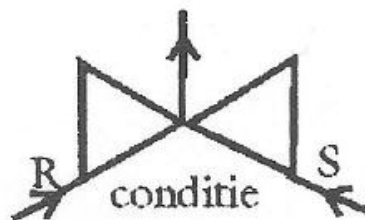
NATURAL JOIN



θ -JOIN

SEMI-JOIN

OUTER-JOIN



Concluzie

Expresia algebrică și arborele algebric nu sunt moduri în care se pot implementa cereri în SQL, ci moduri în care **gândim și înțelegem cum operează serverul**.

Acestea descriu pașii logici prin care datele sunt filtrate, reduse și combinate.

Atunci când se implementează o interogare SQL, se spune *ce rezultat se dorește*, iar serverul (de exemplu Oracle Database) o transformă intern într-o **expresie algebrică**, pe care apoi o reorganizează într-un **arbore algebric optimizat**.

În acest arbore, operațiile precum selecția (filtrarea), proiecția (alegerea coloanelor) și join-ul sunt mutate și reordonate astfel încât să reducă volumul de date cât mai devreme și să minimizeze costul execuției.

Concluzie

De aceea nu trebuie ca implementarea cererii în SQL să fie „în ordinea algebrei”, ci să înțelegem această logică pentru a anticipa cum va optimiza serverul interogarea.

În practică, se formulează cererea SQL, iar serverul o rescrie inteligent într-o formă echivalentă, dar mai eficientă.

Astfel, algebra relațională devine un instrument de gândire și verificare, iar arborele algebric este „planul real de execuție” după care baza de date lucrează efectiv.

Exemplu

SQL inițial:

```
SELECT e.last_name  
FROM employees e  
JOIN departments d ON e.department_id = d.department_id  
WHERE upper(e.last_name) like 'A%' AND d.department_id > 50;
```

Intuitiv, se observă prima dată execuția operației JOIN, urmată de filtrare (WHERE)

Expresia algebrică (neoptimizată) :

```
PROJECT (  
  SELECT (  
    JOIN(employees, departments, department_id),  
    upper(last_name) LIKE 'A%' AND department_id >  
50  
  ),  
  last_name  
)
```

Exemplu

Expresia algebrică (neoptimizată) :

```
PROJECT (  
    SELECT (  
        JOIN (employees, departments, department_id),  
        upper(last_name) LIKE 'A%' AND department_id >  
50  
    ),  
    last_name  
)
```

Expresia algebrică (optimizată) :

```
PROJECT (  
    JOIN (  
        SELECT (employees, upper(last_name) LIKE 'A%'),  
        SELECT (departments, department_id > 50),  
        department_id  
    ),  
    last_name  
)
```

Exemplu

SQL rescris (care reflectă optimizarea):

```
SELECT e.last_name
FROM (SELECT * FROM employees WHERE
upper(last_name) LIKE 'A%') e
JOIN (SELECT * FROM departments WHERE
department_id > 50) d
ON e.department_id = d.department_id;
```

Cum execută de fapt serverul (Oracle Database)

1. Citește employees și păstrează doar angajații cu last_name LIKE 'A%'
2. Citește departments și păstrează doar department_id > 50
3. Face JOIN pe seturi deja reduse
4. Returnează doar last_name

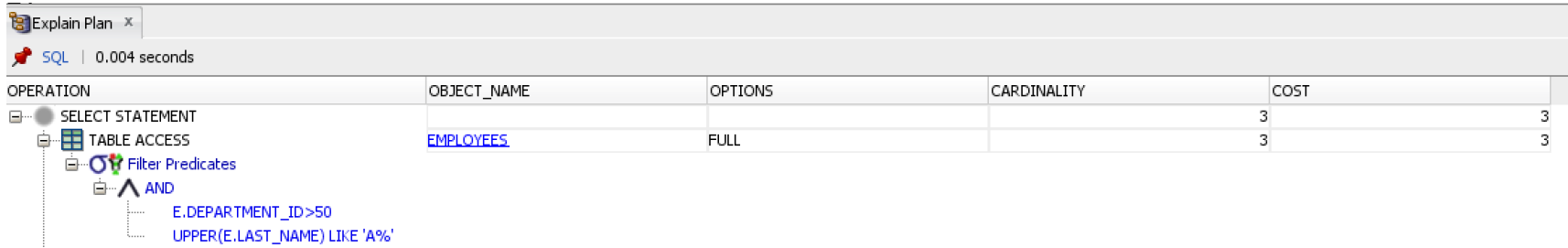
Concluzie

SQL-ul spune **ce dorim să implementăm**.

Algebra relațională arată **cum poate fi executat eficient**.

Serverul transformă automat interogarea într-un arbore optimizat, folosind exact regulile studiate în cadrul acestui capitol.

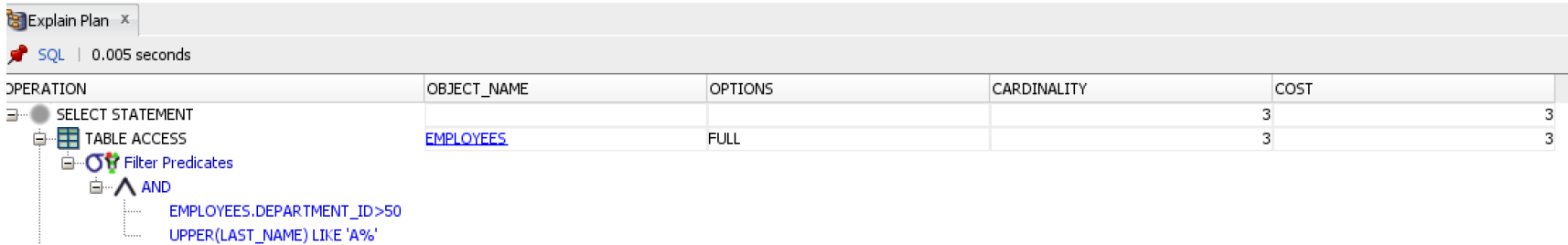
Plan de execuție – query inițial



SQL | 0.004 seconds

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT				3
TABLE ACCESS	EMPLOYEES	FULL		3
Filter Predicates				
AND				
E.DEPARTMENT_ID > 50				
UPPER(E.LAST_NAME) LIKE 'A%'				

Plan de execuție – query optimizat

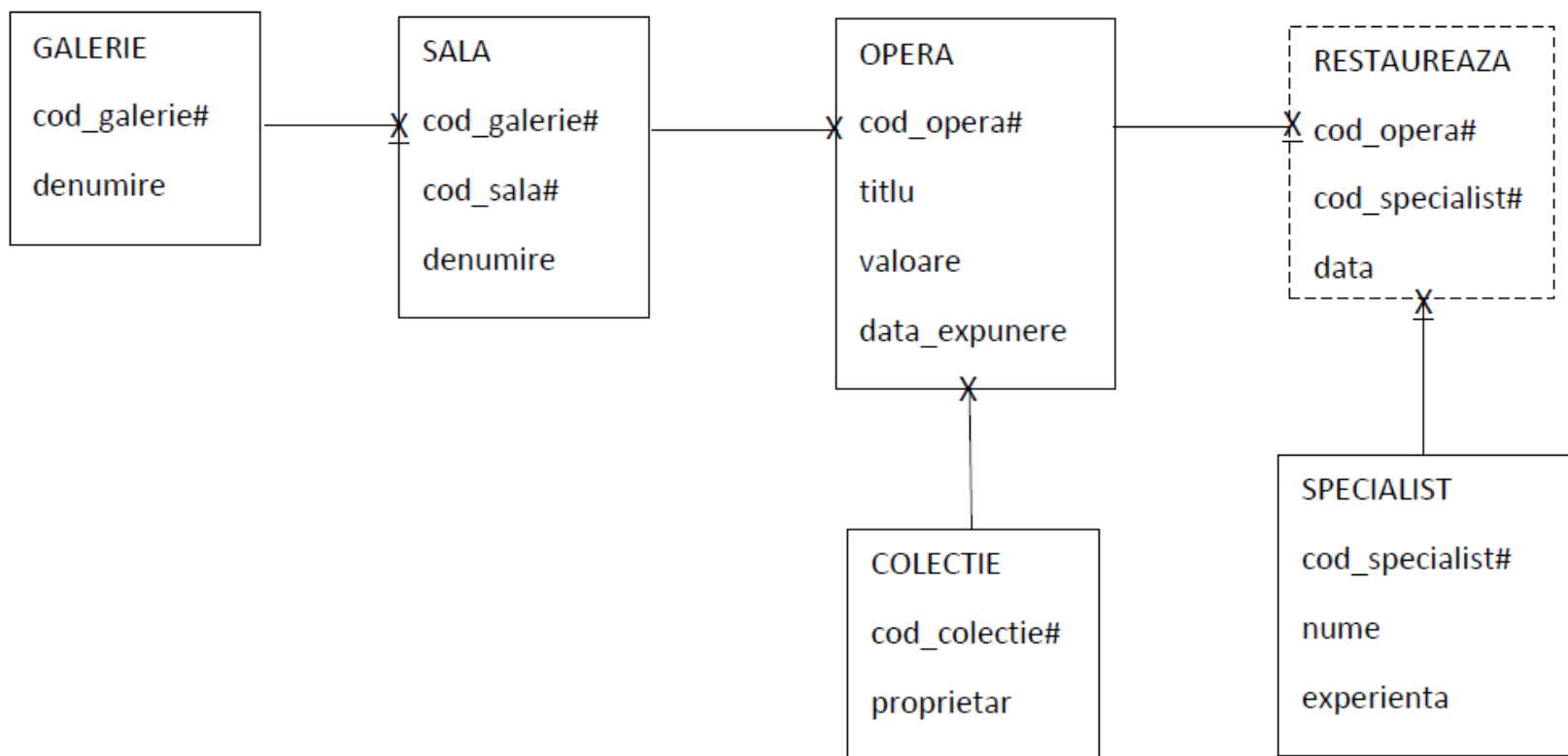


SQL | 0.005 seconds

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT				3
TABLE ACCESS	EMPLOYEES	FULL		3
Filter Predicates				
AND				
EMPLOYEES.DEPARTMENT_ID > 50				
UPPER(LAST_NAME) LIKE 'A%'				

Exemplu

- Să se obțină **titlul** și **valoarea** operelor de artă, expuse în galeria având codul G1, care fac parte din colecțiile ce aparțin proprietarului cu numele King și care au fost restaurate după data 15/06/2000.



Exemplu

Cerere SQL :

```
SELECT titlu, valoare  
FROM opera o JOIN colectie c ON (o.cod_colectie =  
c.cod_colectie)  
JOIN restaureaza r ON (o.cod_opera = r.cod_opera)  
WHERE upper(o.cod_galerie) = 'G1'  
AND initcap(c.proprietar) = 'King'  
AND r.data > to_date('05/06/2000', 'dd/mm/yyyy');
```

Exemplu

Expresie algebrică:

R1 = SELECT(OPERA, cod_galerie = 'G1')

R2 = PROJECT(R1, cod_opera, titlu, valoare, cod_colectie)

R3 = SELECT(COLECTIE, proprietar = 'King')

R4 = PROJECT(R3, cod_colectie)

R5 = SEMIJOIN(R2, R4, cod_colectie)

R6 = SELECT(RESTAUREAZA, data > 05/06/2000)

R7 = PROJECT(R6, cod_opera)

R8 = SEMIJOIN(R5, R7, cod_opera)

Rezultat = R9 = PROJECT(R8, titlu, valoare)

Exemplu

Arbore algebric:

- Arborele algebric se află pe slide-ul următor, dar și în documentul numit: **Arbore_Algebric_Curs8**

R1 = SELECT(OPERA, cod_galerie = 'G1')

R2 = PROJECT(R1, cod_opera, titlu, valoare, cod_colectie)

R3 = SELECT(COLECTIE, proprietar = 'King')

R4 = PROJECT(R3, cod_colectie)

R5 = SEMIJOIN(R2, R4, cod_colectie)

R6 = SELECT(RESTAUREAZA, data > 05/06/2000)

R7 = PROJECT(R6, cod_opera)

R8 = SEMIJOIN(R5, R7, cod_opera)

Rezultat = R9 = PROJECT(R8, titlu, valoare)

